Pipeline CI/CD con Aseguramiento de Calidad Integrado



Wilson Alexander Caal Pacay

Aseguramiento de la Calidad del Software · Ing. Edgar Matul García

Universidad Mariano Gálvez de Guatemala · Noviembre de 2026

Todo cambio se verifica antes de entrar a main

Objetivo

Implementar un pipeline CI/CD con controles automatizados de calidad sobre una aplicación existente, y medir su efecto con métricas de antes y después.

Aplicación base: CRUD de tutoriales con React, Express y MySQL, de bezkoder.

Dentro del alcance

- Plan de aseguramiento de la calidad (SQA Plan)
- Pruebas unitarias, de integración y de regresión
- Análisis estático y quality gates que bloquean el merge
- Rama principal protegida, con pull request obligatorio
- Corrección de defectos e issues, con métricas comparadas
- Docker Compose, variables de entorno y avisos por Telegram

Fuera del alcance

- Desarrollar la aplicación desde cero
- Pruebas de carga, estrés o rendimiento
- Migrar de framework o agregar autenticación

Stack tecnológico por capa



Aplicación
React 18, Express 4, Sequelize 6 y MySQL 8.4



CI/CD
GitHub y GitHub Actions, con Node 24



Pruebas
Jest, Supertest y React Testing Library



Cobertura
Integrada en Jest, con reporte LCOV a SonarCloud



Análisis estático
ESLint, Prettier y SonarCloud



Dependencias
npm audit en el servidor y en el cliente



Contenedores
Docker Compose, solo para MySQL



Notificaciones
Telegram, en cada pull request y en main

Cuatro etapas, y solo se fusiona con los gates en verde

Se dispara con push a cualquier rama · pull request a main · ejecución manual

build

Instala dependencias

Compila el cliente con CI=true

Bloquea

test

Jest, Supertest y MySQL

React Testing Library

Umbral: 70 %, ramas 60 %

Bloquea

analyze

ESLint y Prettier

npm audit del servidor

Quality gate de SonarCloud

Bloquea

report

Resumen de la corrida

Aviso por Telegram

Corre aunque fallen las demás

Informa

Protección de main

Merge solo por pull request, con la rama al día y con build, test, analyze y SonarCloud Code Analysis en verde. Un push directo a main se rechaza, también para el administrador.

Línea base: sin pruebas y con 23 issues

0

pruebas automatizadas

0.0 %

de cobertura

23

issues en SonarCloud

11

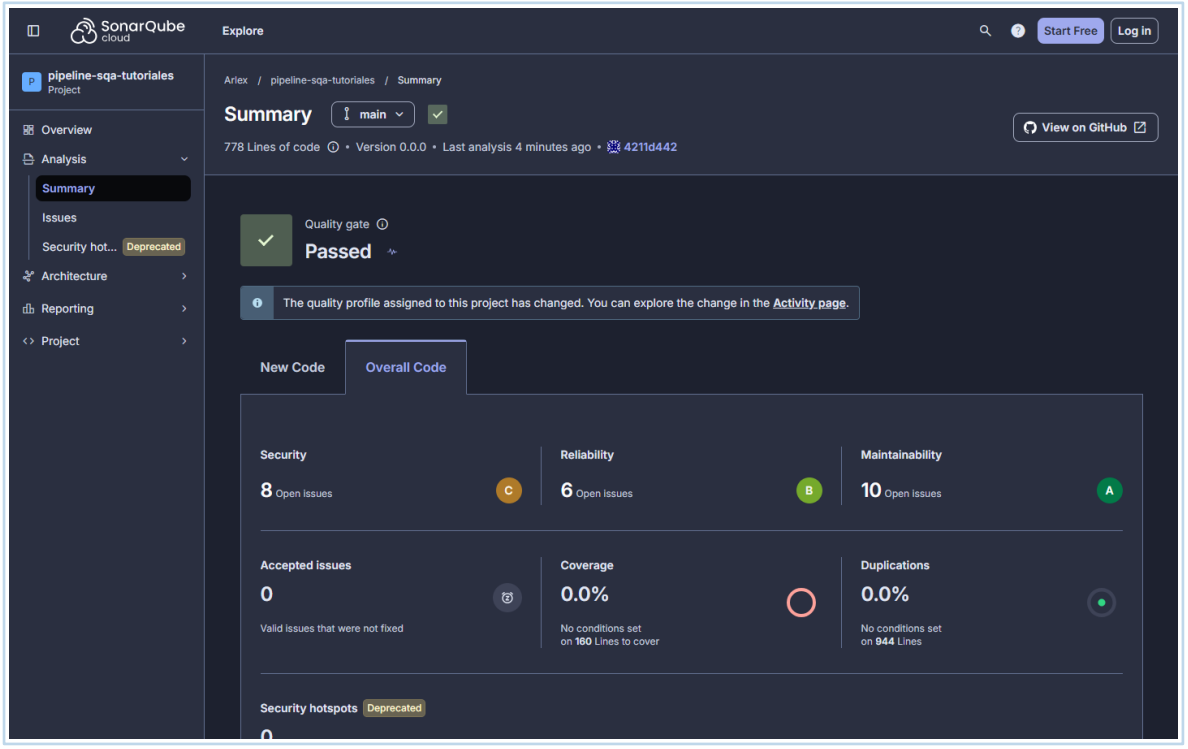
hallazgos de ESLint

1 y 14

avisos críticos y altos de npm audit: servidor y cliente

C · B · A

seguridad, fiabilidad y mantenibilidad

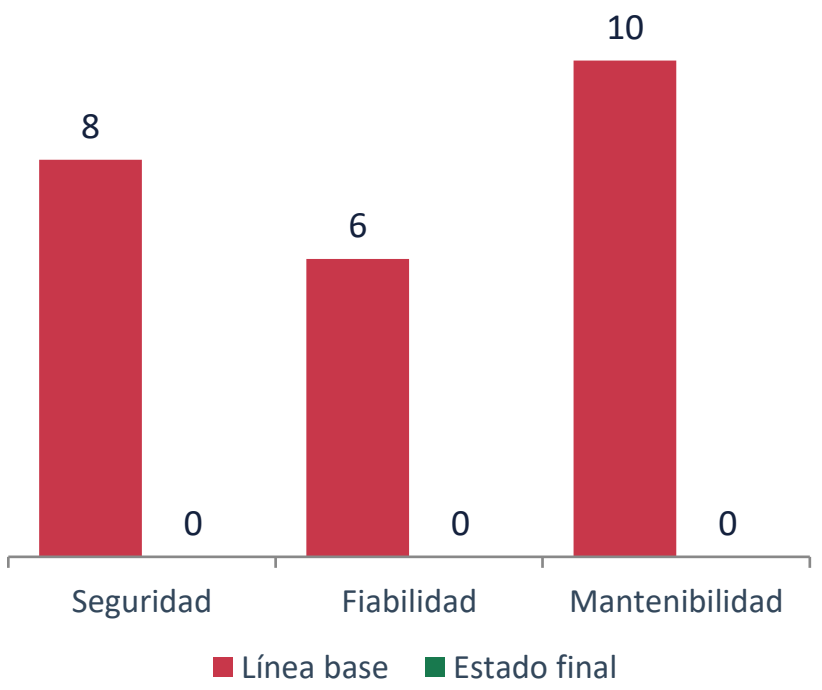


Deuda de mantenibilidad: 1 h 10 min
Remediación total: 4 h 50 min · Duplicación: 0.0 %

Medido sobre v0-linea-base, commit 4211d44. Detalle en docs/Linea-Base-Calidad.md.

Estado final: 48 pruebas y 0 issues en SonarCloud

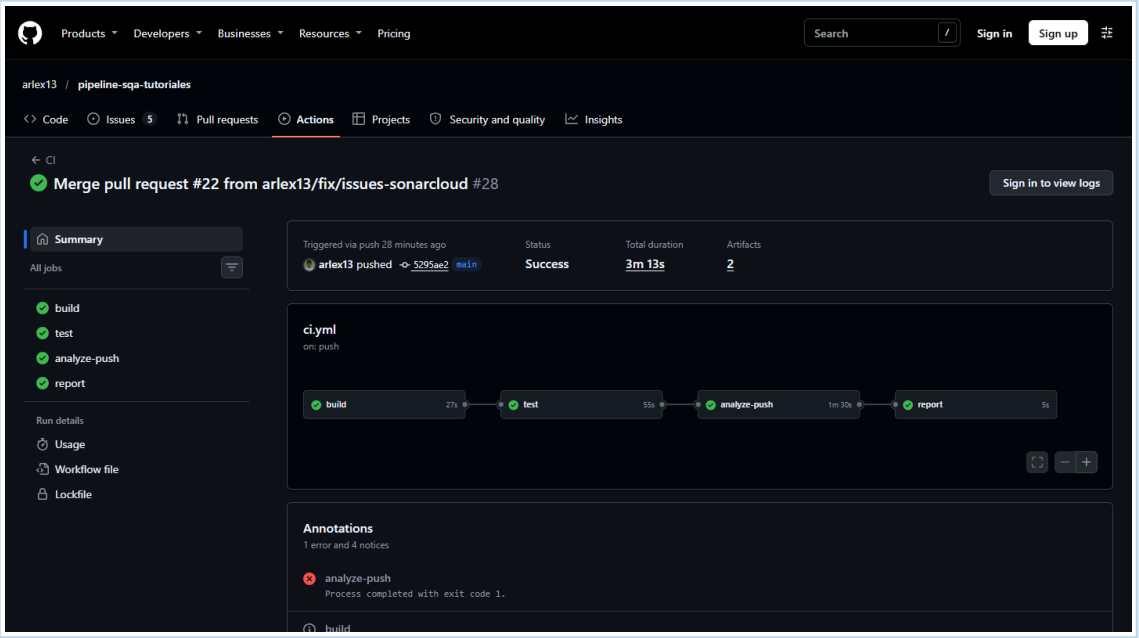
Métrica	Antes	Después
Pruebas automatizadas	0	**48**
Cobertura en SonarCloud	0.0 %	**97.5 %**
Issues de SonarCloud	23	**0**
Seguridad	C	**A**
Fiabilidad	B	**A**
Mantenibilidad	A	**A**
Hallazgos de ESLint	11	**0**
Avisos críticos y altos, servidor	1	**0**
Avisos críticos y altos, cliente (D-12)	14	**13**
Deuda de mantenibilidad	1 h 10 min	**0 min**
Duplicación	0.0 %	**0.0 %**
Quality gate de main	Sin código nuevo	**Aprobado**



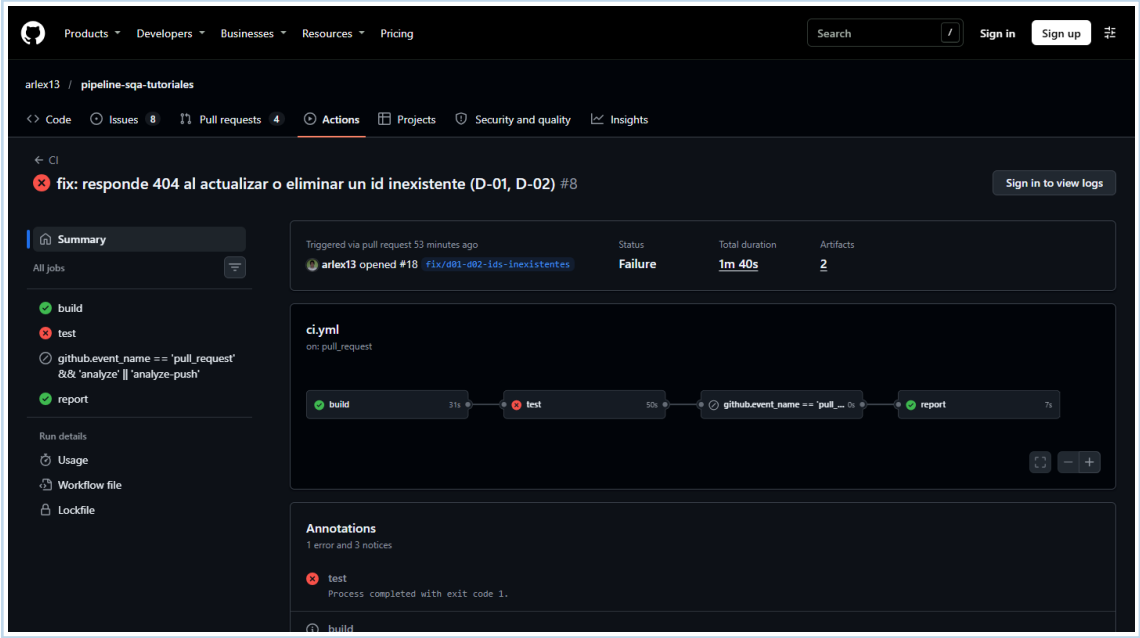
Medido sobre v1.0.0, commit 5295ae2, con las mismas reglas. Las barras suman 24 porque un issue afecta a dos cualidades. Detalle en docs/Reporte-Mejora.md.

El pipeline aprueba lo correcto y rechaza lo que falla

En verde: las cuatro etapas en main



En rojo: una regresión detiene el merge

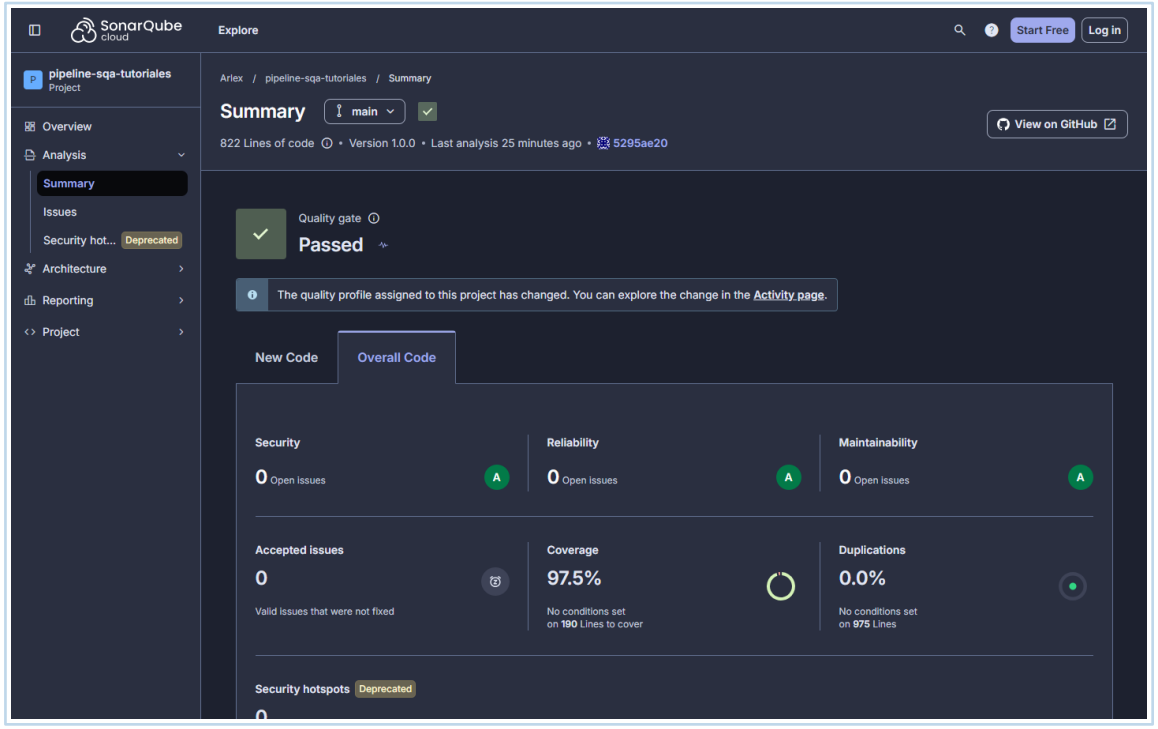


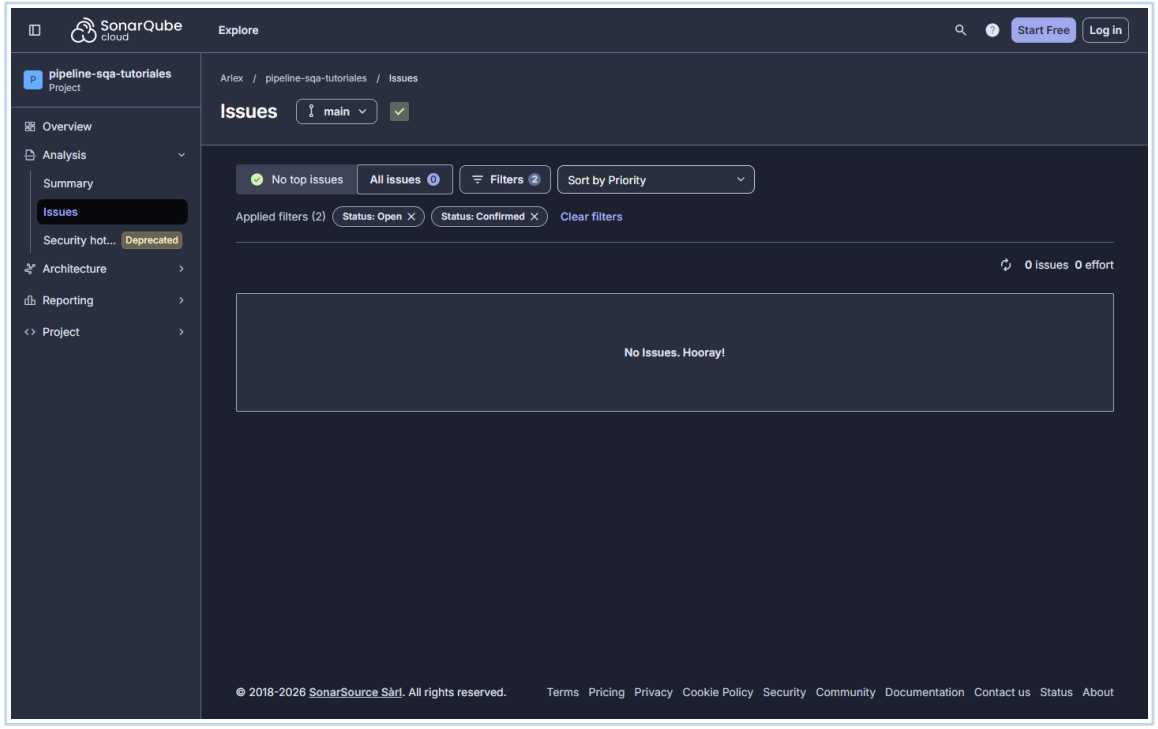
Rechazos reales, con el merge bloqueado por la protección de main:

- ✕ Pruebas: #18 y #19
- ✕ SonarCloud: #20
- ✕ ESLint: #21
- ✕ Push directo: GH006

El panel de SonarCloud confirma el resultado en main

Aprobado	97.5 %	0	0	0.0 %
Quality gate	Cobertura	Bugs	Code smells	Duplicación





Conclusiones y lecciones aprendidas

Lecciones

Un gate vale si se ve fallar.
Pruebas, ESLint y SonarCloud se comprobaron con corridas en rojo y el merge bloqueado.

Las herramientas no lo detectan todo.
D-05, D-14 y D-15 no los marcó SonarCloud: salieron de leer el código y usar la aplicación.

El antes y el después deben medirse igual.
El perfil se eligió antes de medir, y la auditoría se repitió el mismo día.

La revisión manual no se automatiza.
GitHub no la exige sin aprobaciones, y en cuatro pull requests el comentario llegó después del merge.

Dificultades y cómo se resolvieron

Un PUT sin cambios parecía un id inexistente.
MySQL informa 0 filas si los mismos datos llegan en el mismo segundo. Se consulta findByPk antes de responder 404.

Un análisis automático con otro perfil.
SonarCloud lo lanzó al importar el proyecto. Se apagó y se descartó: la línea base es la versión 0.0.0.

El formato no cubría lo mismo que un clon limpio.
Faltaban 3 archivos en los patrones. Se ampliaron y se repitió la medición: 19 de 24.

Una cifra salió mal en el archivo de medición.
Bash le quitó las barras a una expresión regular. Se detectó al revisar la salida y se corrigió con otro commit.

Referencias

Este proyecto

- Repositorio: github.com/arlex13/pipeline-sqa-tutoriales
- Panel de SonarCloud: arlex13_pipeline-sqa-tutoriales
- Documentos en docs/: SQA-Plan, Linea-Base-Calidad, Matriz-Pruebas, Reporte-Mejora, Manual-Tecnico y Manual-Uso
- Aplicación base; su servidor declara licencia ISC: github.com/bezkoder/react-express-mysql

Normas y documentación

- IEEE 730: procesos de aseguramiento de la calidad del software
- ISO/IEC 25010: modelo de calidad del producto
- ISO/IEC/IEEE 29119: pruebas de software
- Documentación de GitHub Actions y de SonarQube Cloud
- Documentación de Jest, Supertest, React Testing Library, ESLint y Prettier